

AI-Architect Architecture Report

Project: KIconnect Brownfield

Run ID: 20260828T144501Z-b2c5ad32

Status: Accepted — 2026-08-28 14:53 UTC

Final Review Verdict: PASS

Models used: gemini-3.1-flash-lite, OpenAI GPT OSS 120b KI:Inferenz.nrw

Executive Summary

Online customers interact with a responsive React SPA that communicates via a unified API Gateway. Customer support, warehouse staff, finance, and product management gain faster feature delivery because each business capability (user accounts, catalog, orders, payments, reviews) is owned by a dedicated service that can be scaled, updated, and deployed independently. Operations staff benefit from automated scaling, health-checks, and clear ownership of services, reducing operational risk during traffic spikes.

Requirements & Constraints

Business Goal

Modernize the existing e-commerce monolith so it can scale during peak traffic, remain maintainable as the business grows, and evolve incrementally without disrupting ongoing operations.

Problem Statement

The current monolith has scalability and maintainability problems, especially during peak traffic. Changes are difficult to deploy safely, and tightly coupled functionality makes independent evolution difficult.

Users / Stakeholders

- Online customers
- customer support
- warehouse and inventory staff
- finance and payment operations
- product management
- software engineers
- operations staff

Functional Requirements

- User account management
- Product browsing and management
- Order processing
- Payment integration
- Product review functionality

Non-Functional Requirements

- Support up to 50,000 concurrent users during peak campaigns.
- Maintain high availability during peak periods.
- Keep customer-facing interactions responsive.

- Allow the system to scale as demand grows.
- Support incremental migration without major service disruption.

Cloud Provider

AWS

Budget

No fixed budget; focus on cost efficiency

Existing Systems

- Django backend
- React frontend
- Python based monolith

Recommended Architecture

Pattern

Strangler Fig (incremental microservices extraction)

Rationale

The Strangler Fig pattern enables incremental extraction of capabilities from the existing Django monolith into independent services while preserving continuous operation. This aligns with the business goal of scaling during peak traffic, improving maintainability, and supporting zero-downtime evolution on AWS.

Components

ID	Name	Type	Purpose
COMP-001	Legacy Monolith	service	Existing Django monolith that currently implements all e-commerce capabilities.
COMP-002	API Gateway	gateway	Unified entry point for all client traffic, enabling routing, feature-flag based traffic shifting, and security enforcement.
COMP-003	User Service	service	Manage user registration, authentication, profile data, and related events.
COMP-004	Product Service	service	Provide product catalog management and search capabilities.
COMP-005	Order Service	service	Process customer orders, manage order lifecycle, and coordinate with payment and inventory.
COMP-006	Payment Service	service	Integrate with external payment gateways and record transaction details.
COMP-007	Review Service	service	Collect, moderate, and display product reviews.
COMP-008	Frontend (React SPA)	ui	Provide the customer-facing web interface for browsing, shopping, and account management.
COMP-009	PostgreSQL Cluster	database	Persist relational data for all domain services.
COMP-010	Message Queue (Amazon SQS)	queue	Provide reliable asynchronous messaging between services.
COMP-011	Event Bus (Amazon SNS)	bus	Broadcast domain events to multiple interested services.

COMP-001: Legacy Monolith

The current codebase provides user, product, order, payment, and review functionality. It will be gradually decomposed while remaining operational behind the API Gateway.

Technology

Python 3.x, Django, Docker

Traces to features

FEAT-001, FEAT-002, FEAT-003, FEAT-004, FEAT-005, FEAT-010

Justified by

ADR-005

COMP-002: API Gateway

Deployed on AWS API Gateway (or an open-source alternative) to forward requests to either the Legacy Monolith or the newly extracted services. Handles TLS termination and request authentication.

Technology

AWS API Gateway, Lambda authorizers for auth

Traces to features

FEAT-001, FEAT-002, FEAT-003, FEAT-004, FEAT-005, FEAT-008, FEAT-010

Justified by

ADR-003, ADR-004, ADR-005

COMP-003: User Service

A Django application exposing REST endpoints for user CRUD, login, and profile updates. Owns the "user" schema in PostgreSQL and publishes UserCreated/Updated events.

Technology

Python 3.x, Django REST Framework, Docker, AWS ECS/Fargate

Traces to features

FEAT-001

Justified by

ADR-001, ADR-002, ADR-006

COMP-004: Product Service

Handles product CRUD, image storage references, and search indexing. Owns the "product" schema.

Technology

Python 3.x, Django REST Framework, Elasticsearch (optional for search), AWS ECS/Fargate

Traces to features

FEAT-002

Justified by

ADR-001, ADR-002, ADR-006

COMP-005: Order Service

Exposes order creation, status tracking, and order history APIs. Owns the "order" schema and participates in event-driven workflows (OrderCreated, PaymentProcessed).

Technology

Python 3.x, Django REST Framework, AWS ECS/Fargate

Traces to features

FEAT-003

Justified by

ADR-001, ADR-002, ADR-003, ADR-006

COMP-006: Payment Service

Handles payment initiation, callback processing, and stores payment status. Owns the "payment" schema and emits PaymentProcessed events.

Technology

Python 3.x, Django REST Framework, AWS ECS/Fargate, Third-party payment SDKs

Traces to features

FEAT-004

Justified by

ADR-001, ADR-002, ADR-003, ADR-006

COMP-007: Review Service

Provides endpoints for creating and retrieving reviews. Owns the "review" schema and publishes ReviewCreated events.

Technology

Python 3.x, Django REST Framework, AWS ECS/Fargate

Traces to features

FEAT-005

Justified by

ADR-001, ADR-002, ADR-006

COMP-008: Frontend (React SPA)

A React single-page application consuming the API Gateway endpoints. Built with Redux for state management.

Technology

React, Redux, Axios, AWS S3 for static hosting

Traces to features

FEAT-001, FEAT-002, FEAT-003, FEAT-004, FEAT-005, FEAT-008

Justified by

ADR-003, ADR-004, ADR-006

COMP-009: PostgreSQL Cluster

A managed Amazon RDS PostgreSQL instance with Multi-AZ deployment. Each service uses its own schema, enforcing data ownership.

Technology

Amazon RDS PostgreSQL, Read replicas for scaling reads

Traces to features

FEAT-001, FEAT-002, FEAT-003, FEAT-004, FEAT-005

Justified by

ADR-002, ADR-004

COMP-010: Message Queue (Amazon SQS)

Standard SQS queues used for event propagation (e.g., OrderCreated, PaymentProcessed).

Supports at-least-once delivery and decouples producers from consumers.

Technology

Amazon SQS Standard queues, Dead-letter queues for error handling

Traces to features

FEAT-006, FEAT-007, FEAT-009, FEAT-010

Justified by

ADR-003, ADR-004

COMP-011: Event Bus (Amazon SNS)

SNS topics used for fan-out of events such as OrderCreated, PaymentProcessed, and ReviewCreated, enabling multiple services to react.

Technology

Amazon SNS topics, Subscription to SQS queues

Traces to features

FEAT-006, FEAT-007, FEAT-009, FEAT-010

Justified by

ADR-003, ADR-004

Data Flows

- Frontend (React SPA) → API Gateway
- API Gateway → User Service
- API Gateway → Product Service
- API Gateway → Order Service
- API Gateway → Payment Service
- API Gateway → Review Service
- API Gateway → Legacy Monolith
- User Service → PostgreSQL Cluster
- Product Service → PostgreSQL Cluster
- Order Service → PostgreSQL Cluster
- Payment Service → PostgreSQL Cluster
- Review Service → PostgreSQL Cluster
- Legacy Monolith → PostgreSQL Cluster
- Order Service → Message Queue (Amazon SQS)
- Payment Service → Message Queue (Amazon SQS)
- Message Queue (Amazon SQS) → Order Service
- Message Queue (Amazon SQS) → Payment Service
- Order Service → Event Bus (Amazon SNS)
- Payment Service → Event Bus (Amazon SNS)
- Event Bus (Amazon SNS) → Order Service
- Event Bus (Amazon SNS) → Payment Service

Architecture Decision Records

ADR-001: ADR-001: Service decomposition using Domain-Driven Design

Status: accepted

Context

The monolith tightly couples user, product, order, payment, and review capabilities, hindering independent scaling and evolution.

Decision

Define bounded contexts aligned with business capabilities and implement each as an independent Django service.

Rationale

DDD aligns service boundaries with natural domain divisions, reducing coupling and enabling independent deployment, which directly addresses scalability and maintainability goals.

Alternatives Considered

- Keep a modular monolith
- Extract services without domain analysis

Positive Consequences

- Clear ownership per capability
- Independent scaling and deployment
- Improved team autonomy

Negative Consequences / Trade-offs

- Increased operational complexity
- Need for data synchronization during migration

Related Features: FEAT-001, FEAT-002, FEAT-003, FEAT-004, FEAT-005, FEAT-010

Related Components: User Service, Product Service, Order Service, Payment Service, Review Service

Related Decision Topics: TOPIC-1

Evidence: KB-E001, KB-E002

ADR-002: ADR-002: Service-specific data ownership

Status: accepted

Context

Shared database tables across domains cause tight coupling and hinder independent evolution.

Decision

Each service owns its own schema within a shared PostgreSQL cluster; no service accesses another's tables directly.

Rationale

Service-owned data enforces boundaries, simplifies migrations, and supports independent scaling, matching the DDD approach.

Alternatives Considered

- Single shared schema for all services
- Separate database instances per service

Positive Consequences

- Data isolation per domain
- Simpler schema evolution

Negative Consequences / Trade-offs

- Potential duplication of reference data
- Need for cross-service data composition

Related Features: FEAT-001, FEAT-002, FEAT-003, FEAT-004, FEAT-005, FEAT-010

Related Components: User Service, Product Service, Order Service, Payment Service, Review Service, PostgreSQL Cluster

Related Decision Topics: TOPIC-2

Evidence: KB-E003

ADR-003: ADR-003: Hybrid integration (sync UI, async events)

Status: accepted

Context

User-facing interactions require low-latency responses, while internal workflows benefit from decoupling.

Decision

Use synchronous HTTP for UI-driven calls via API Gateway and asynchronous event-driven communication (SNS/SQS) for inter-service processes.

Rationale

Combines responsive user experience with scalable, fault-tolerant backend processing, as recommended for e-commerce workloads.

Alternatives Considered

- Fully synchronous REST calls
- Fully asynchronous event-driven architecture

Positive Consequences

- Fast UI responses
- Loose coupling and better fault isolation

Negative Consequences / Trade-offs

- Added complexity of managing queues and eventual consistency

Related Features: FEAT-006, FEAT-007, FEAT-008, FEAT-009, FEAT-010

Related Components: API Gateway, Message Queue (Amazon SQS), Event Bus (Amazon SNS)

Related Decision Topics: TOPIC-3

Evidence: KB-E004, KB-E005

ADR-004: ADR-004: AWS auto-scaling and multi-AZ deployment

Status: accepted

Context

Peak traffic of up to 50,000 concurrent users requires elastic capacity and high availability.

Decision

Deploy services in AWS ECS/Fargate with auto-scaling groups and place them behind an Application Load Balancer across multiple Availability Zones. Use RDS Multi-AZ for PostgreSQL.

Rationale

AWS managed scaling and multi-AZ provide the elasticity and resilience needed without manual capacity planning, directly supporting the required scalability and availability targets.

Alternatives Considered

- Fixed-size EC2 instances
- Self-managed Kubernetes cluster

Positive Consequences

- Automatic response to load spikes
- Reduced downtime risk

Negative Consequences / Trade-offs

- Potential cost increase if scaling is not tuned

Related Features: FEAT-006, FEAT-007, FEAT-008, FEAT-009

Related Components: API Gateway, User Service, Product Service, Order Service, Payment Service, Review Service, PostgreSQL Cluster, Message Queue (Amazon SQS)

Related Decision Topics: TOPIC-4

Evidence: KB-E005, KB-E009

ADR-005: ADR-005: Incremental migration via Strangler Fig

Status: accepted

Context

The need to modernize without disrupting ongoing operations.

Decision

Introduce a façade (API Gateway) and migrate capabilities one bounded context at a time, routing traffic via feature flags until the monolith can be retired.

Rationale

Strangler Fig enables zero-downtime migration, aligns with the incremental migration requirement, and leverages existing AWS routing capabilities.

Alternatives Considered

- Big-bang rewrite
- Parallel run without façade

Positive Consequences

- Continuous service availability
- Gradual risk exposure

Negative Consequences / Trade-offs

- Longer overall migration timeline
- Complexity of dual-write and routing logic

Related Features: FEAT-010, FEAT-006, FEAT-007, FEAT-008, FEAT-009

Related Components: Legacy Monolith, API Gateway, User Service, Product Service, Order Service, Payment Service, Review Service

Related Decision Topics: TOPIC-5

Evidence: KB-E006, KB-E009

ADR-006: ADR-006: Preserve existing Python/Django and React stack

Status: accepted

Context

The repository already uses Python/Django for backend and React for frontend; no compelling requirement to replace them.

Decision

Retain Python/Django for all new services and keep the React SPA unchanged, adding only AWS managed infrastructure where needed.

Rationale

Reusing the existing stack minimizes rewrite effort, leverages team expertise, and reduces risk, while still achieving scalability via cloud services.

Alternatives Considered

- Rewrite services in a different language/framework
- Adopt a polyglot microservices approach

Positive Consequences

- Lower development cost
- Faster migration due to familiar codebase

Negative Consequences / Trade-offs

- Potential limitations of Django for certain high-throughput scenarios

Related Features: FEAT-001, FEAT-002, FEAT-003, FEAT-004, FEAT-005, FEAT-006, FEAT-007, FEAT-008, FEAT-009, FEAT-010

Related Components: Legacy Monolith, User Service, Product Service, Order Service, Payment Service, Review Service, Frontend (React SPA)

Related Decision Topics: TOPIC-6

Evidence: KB-E007

Migration Plan

Step 1: Introduce API Gateway and routing layer

Objective

Create a façade that abstracts the monolith and new services, enabling traffic routing via feature flags.

Changes

- Deploy AWS API Gateway
- Configure routes to Legacy Monolith for all existing endpoints

Coexistence / Data Strategy

All external traffic passes through the gateway; routing rules initially point to the monolith.

Exit Condition

Gateway health checks pass and traffic is successfully routed.

Step 2: Extract User Service

Objective

Move user account management to an independent service.

Changes

- Create User Service (Django app)

- Migrate user tables to dedicated schema
- Update API Gateway routes for /users to User Service

Coexistence / Data Strategy

Dual-write during migration; feature flag gradually shifts traffic to User Service.

Exit Condition

All user-related API calls are served by User Service and tests pass.

Step 3: Extract Product Service

Objective

Isolate product catalog management.

Changes

- Create Product Service
- Migrate product tables to its schema
- Route /products endpoints to Product Service

Coexistence / Data Strategy

Feature flag controls traffic split; data sync scripts keep legacy and new tables consistent.

Exit Condition

Product API fully served by Product Service.

Step 4: Extract Order Service with event-driven workflow

Objective

Decouple order processing and enable asynchronous handling.

Changes

- Create Order Service
- Migrate order tables
- Publish OrderCreated events to SNS/SQS
- Consume payment events for order state updates

Coexistence / Data Strategy

Orders created via gateway are persisted in both monolith and Order Service during cut-over.

Exit Condition

Order lifecycle managed entirely by Order Service.

Step 5: Extract Payment Service

Objective

Separate payment integration.

Changes

- Create Payment Service
- Migrate payment tables
- Publish PaymentProcessed events

Coexistence / Data Strategy

Feature flag routes payment calls; events synchronize order status.

Exit Condition

All payment interactions handled by Payment Service.

Step 6: Extract Review Service

Objective

Isolate product review functionality.

Changes

- Create Review Service
- Migrate review tables
- Update gateway routes for /reviews

Coexistence / Data Strategy

Dual-write and eventual consistency for existing reviews.

Exit Condition

Review API fully served by Review Service.

Step 7: Decommission Legacy Monolith

Objective

Retire the original monolith after all capabilities are migrated.

Changes

- Remove monolith routes from API Gateway
- Shut down monolith containers
- Archive legacy codebase

Coexistence / Data Strategy

All traffic now handled by extracted services; data resides in service-specific schemas.

Exit Condition

No requests are routed to Legacy Monolith and monitoring confirms zero errors.

Evidence / Literature

Curated Knowledge Base Evidence

ID	Source	Page	Excerpt
KB-E001	Rag Database/box2_domain/ecommerce_migration_event_driven_bulus.md	0	The reviewed studies reveal several distinct approaches to monolith decomposition, each with specific advantages for e-commerce contexts. **a. Domain-Driven Design (DDD) Approaches:** Multiple studies emphasize Domain-Driven Design as a foundational approach for identifying service boundaries in e-commerce systems. A...
KB-E002	Rag Database/box2_domain/ecommerce_migration_event_driven_bulus.md	0	Microservices architecture has emerged as a compelling alternative, offering the promise of enhanced scalability, improved fault tolerance, and greater development agility [1]. By decomposing monolithic applications into smaller, independently deployable services, organizations can achieve better resource utilization,...
KB-E003	Rag Database/box1_patterns/architecture_patterns_v2.md	0	**Solution mechanics:** Each service exclusively owns its data; persistent data is accessed only through that service's API, never by another service reaching into its tables. Implementation variants (increasing isolation): private-tables-per-service, schema-per-service, and database-server-per-service (private-tables...

ID	Source	Page	Excerpt
KB-E004	Rag Database/box2_domain/ecommerce_migration_event_driven_bulus.md	0	**b. High Load Scenarios:** Asynchronous patterns (message queues) demonstrate superior throughput and availability [4]. **c. Availability:** Event-driven architectures provide better fault tolerance and system availability under stress [4]. These findings suggest that e-commerce systems should employ hybrid approach...
KB-E005	Rag Database/box2_domain/ecommerce_microservices_challenges_ibrahim_luong.md	0	## 2.2 Monolithic and Microservices Architectures Comparison *(printed p. 478)* In the e-commerce landscape, choosing between microservices and traditional monolithic architectures requires careful consideration of various factors. Monolithic architectures, characterized by their single, tightly coupled codebase. How...
KB-E006	Rag Database/box2_domain/ecommerce_migration_event_driven_bulus.md	0	# Monolith-to-Microservices-Migration im E-Commerce: Zerlegung, Event-Driven Patterns und Leistung (kuratorischer Auszug) - **Source title:** Migrating Monolithic E-Commerce Systems to Microservices: A Systematic Review of Event-Driven Architecture Approaches (curated excerpts) - **Author(s):** Stephen W. Bulus, Olub...
KB-E007	Rag Database/box2_domain/ecommerce_migration_event_driven_bulus.md	0	## Introduction — relevante Absätze zur E-Commerce-Migration *(printed p. 2)* The rapid evolution of e-commerce platforms has necessitated architectural paradigm shifts to meet increasing demands for scalability, reliability, and agility [2]. Traditional monolithic architectures, while providing simplicity in develop...
KB-E008	Rag Database/box2_domain/ecommerce_polyglot_persistence_microsoft.md	0	With a domain-driven microservices approach, each service uses the database that fits its data characteristics. Each microservice owns its private data store. This design prevents unintentional coupling between services and supports independent updates and deployments without coordinating changes across the system. #...
KB-E009	Rag Database/box1_patterns/microservices-on-aws.pdf	5	Modernizing to microservices Microservices are essentially small, independent units that make up an application. Transitioning from traditional monolithic structures to microservices can follow various strategies. Are you Well-Architected? 2

Validation & Reviewer Findings

Overall Verdict: PASS

Refinement rounds: 1

No open findings were recorded on the accepted review.

Traceability

Features -> Decisions / Components

Feature	Name	ADRs	Components
FEAT-001	User Account Management	ADR-001, ADR-002, ADR-006	COMP-001, COMP-002, COMP-003, COMP-008, COMP-009
FEAT-002	Product Browsing and Management	ADR-001, ADR-002, ADR-006	COMP-001, COMP-002, COMP-004, COMP-008, COMP-009
FEAT-003	Order Processing	ADR-001, ADR-002, ADR-006	COMP-001, COMP-002, COMP-005, COMP-008, COMP-009
FEAT-004	Payment Integration	ADR-001, ADR-002, ADR-006	COMP-001, COMP-002, COMP-006, COMP-008, COMP-009
FEAT-005	Product Review Functionality	ADR-001, ADR-002, ADR-006	COMP-001, COMP-002, COMP-007, COMP-008, COMP-009
FEAT-006	Scalability for Peak Traffic	ADR-003, ADR-004, ADR-005, ADR-006	COMP-010, COMP-011
FEAT-007	High Availability	ADR-003, ADR-004, ADR-005, ADR-006	COMP-010, COMP-011
FEAT-008	Responsive Customer-Facing Interactions	ADR-003, ADR-004, ADR-005, ADR-006	COMP-002, COMP-008
FEAT-009	Elastic Scaling	ADR-003, ADR-004, ADR-005, ADR-006	COMP-010, COMP-011
FEAT-010	Incremental Migration Support	ADR-001, ADR-002, ADR-003, ADR-005, ADR-006	COMP-001, COMP-002, COMP-010, COMP-011

Decision Topics -> ADRs

Topic	Question	ADRs
TOPIC-1	service decomposition and boundaries	ADR-001
TOPIC-2	data ownership and persistence strategy	ADR-002
TOPIC-3	integration style: synchronous vs asynchronous communication	ADR-003
TOPIC-4	scaling and availability strategy	ADR-004
TOPIC-5	brownfield migration and evolution strategy	ADR-005
TOPIC-6	technology conservation vs replacement	ADR-006

ADRs -> Evidence

ADR	Evidence
ADR-001	KB-E001, KB-E002
ADR-002	KB-E003
ADR-003	KB-E004, KB-E005
ADR-004	KB-E005, KB-E009
ADR-005	KB-E006, KB-E009
ADR-006	KB-E007

Limitations / Open Items

No unresolved items are recorded on this run.